LeetCode – Nov 2026 | Day 4

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Nov 10, 2026

Introduction

Today's practice focuses on two **in-place two-pointer** problems: merging two sorted arrays from right to left and moving all zeroes to the end while preserving the relative order of nonzero elements. Both solutions run in linear time and use constant extra space.

Leetcode 88 - Merge Sorted Array

Question

Given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and integers `m` and `n` representing their respective numbers of valid elements, merge them into a single sorted array **in-place** inside `nums1`.

`nums1` has length `m + n`; only its first `m` elements are valid initially. The last `n` positions are placeholders, not values to merge.

Example:

```
Input:  nums1 = [1,2,3,0,0,0], m = 3
        nums2 = [2,5,6], n = 3
Output: nums1 = [1,2,2,3,5,6]
```

Intuition

Merging from the beginning would risk overwriting unprocessed values in `nums1`. Instead, start from the **end**, where `nums1` has enough free positions. Compare the largest remaining elements of each array and place the larger one into the last available position.

Approach

1. Set `i = m - 1` to the last valid element of `nums1`, `j = n - 1` to the last element of `nums2`, and `k = m + n - 1` to the last available slot in `nums1`.
2. While both arrays have remaining elements, place the larger of `nums1[i]` and `nums2[j]` at `nums1[k]`. Move the corresponding source pointer and decrement `k`.
3. If `nums2` still has elements, copy them into the remaining positions of `nums1`.
4. If `nums1` still has elements, leave them in place; they are already correctly positioned.

When values are equal, this implementation chooses the element from `nums2` for the rightmost available position.

Time Complexity

$$\Theta(m+n)$$

Each iteration processes one element, and at most `m + n` elements are processed.

Space Complexity

$$\Theta(1)$$

Only three index variables are used; the merge is performed directly in `nums1`.

Code

```java
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1;
        int j = n - 1;
        int k = m + n - 1;

        while (i >= 0 && j >= 0) {
            if (nums1[i] <= nums2[j]) {
                nums1[k] = nums2[j];
                j--;
                k--;
            } else {
                nums1[k] = nums1[i];
                i--;
                k--;
            }
        }

        while (j >= 0) {
            nums1[k] = nums2[j];
            k--;
            j--;
        }
    }
}
```

Leetcode 283 - Move Zeroes

Question

Given an integer array `nums`, move all zeroes to the end **in-place**, while maintaining the relative order of the nonzero elements. Do not create a copy of the array.

Example:

```
Input:  nums = [0,1,0,3,12]
```

```
Output: nums = [1,3,12,0,0]
```

Intuition

Use a **slow pointer** to track the next position for a nonzero element and a **fast pointer** to scan the array. When the fast pointer finds a nonzero value, swap it into the next available position. Skipping swaps when the pointers are equal avoids unnecessary writes.

Approach

1. Initialize `i = 0` as the next position for a nonzero element.
2. Use `j` to traverse the array from left to right.
3. If `nums[j]` is nonzero, swap `nums[i]` and `nums[j]` **only when** `i != j`.
4. Increment `i` whenever a nonzero value is encountered, regardless of whether a swap occurred.
5. After the traversal, nonzero elements retain their original relative order, and all remaining positions contain zeroes.

Time Complexity

$$\Theta(n)$$

The fast pointer traverses all `n` elements once. Each swap takes constant time.

Space Complexity

$$\Theta(1)$$

Only the pointers and one temporary variable are used, with no additional array.

Code

```java
class Solution {
    public void moveZeroes(int[] nums) {
        int i = 0;

        for (int j = 0; j < nums.length; j++) {
            if (nums[j] != 0) {
                if (i != j) {
                    int temp = nums[i];
                    nums[i] = nums[j];
                    nums[j] = temp;
                }
                i++;
            }
        }
    }
}
```

Summary

Both problems use **two pointers** and modify the original array without allocating extra storage. The key distinction is the direction of traversal:

- **LeetCode 88:** Merge **right to left** to avoid overwriting unprocessed values in `nums1`.
- **LeetCode 283:** Scan **left to right** and move nonzero values forward while preserving their relative order.

Both achieve linear time and constant extra space.